

Partie A: Script Shell et Linux

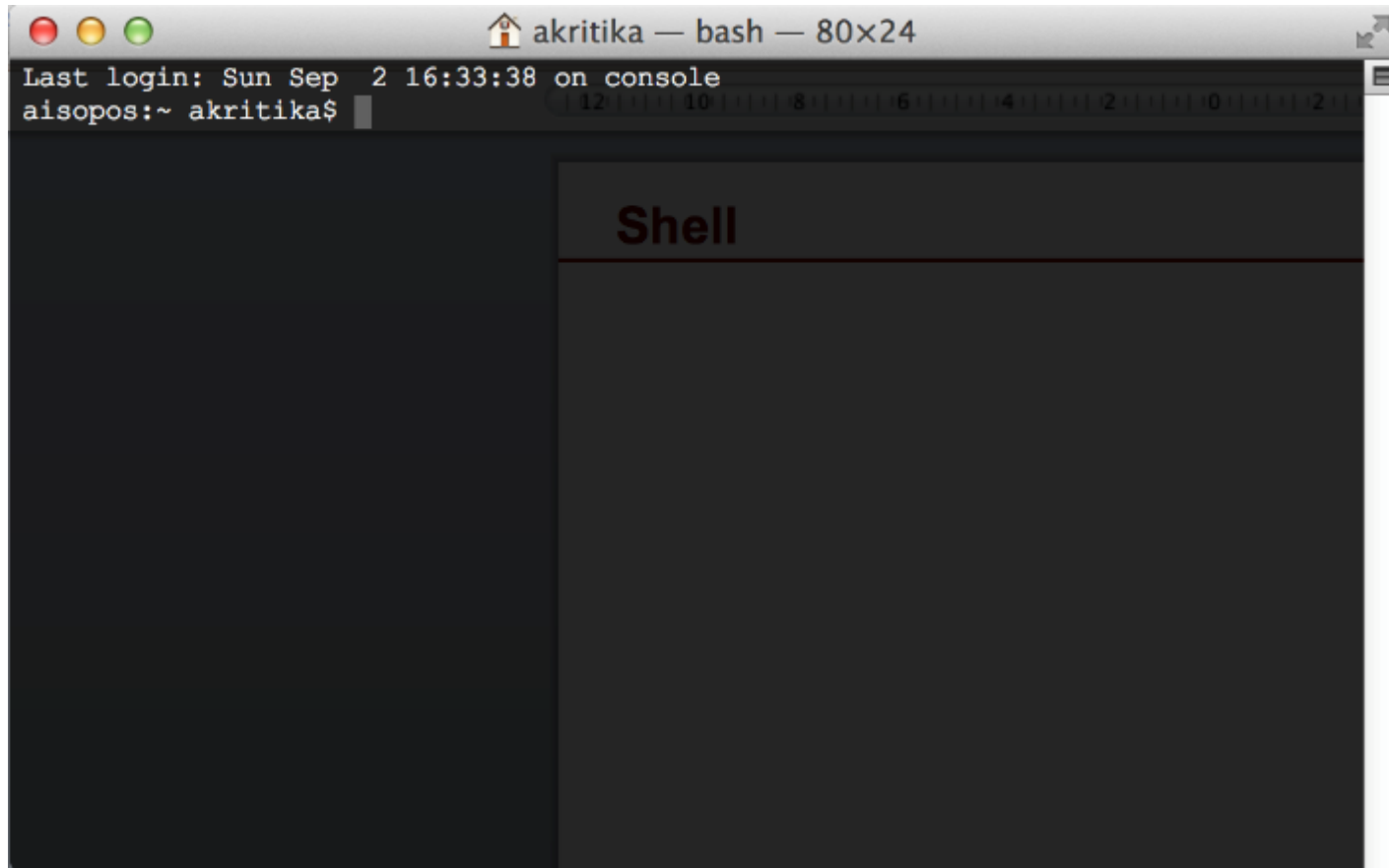
BAKAYOKO Ibrahima
Enseignant-Chercheur
UFR MI
bakayoko.ibrahima1@ufhb.edu.ci

- Gère :
 - Processus : création, exécution, terminaison, ...
 - Mémoire : isolation, pagination, mémoire virtuelle, ...
 - Fichiers : création, accès, autorisations, ...
 - Périphériques : utilisation, interface commune, ...
- Fournit une abstraction de couche supérieure pour interagir avec les périphériques matériels
- Appels système
- Mise au point : Utilisation du système d'exploitation
 - Linux

- Premier prototype en 1991 (posté sur 'comp.os.minix') :
 - Linus Torvalds (étudiant finlandais - 21 ans) :

*“Bonjour à tous ceux qui utilisent Minix -
Je suis en train de créer un système d'exploitation (gratuit) (juste un passe-temps, ne sera pas grand et professionnel comme GNU) pour les clones AT 386 (486)...”*
- Dénomination :
 - Linus : “Freax” → “Free”, “Freak”, “x” (from Unix)
 - Ari Lemmke (administrateur de serveur) : Renommé "Linux"
- Système d'exploitation libre et open source
 - > 18.000.000 lignes de code source

- Ce n'est pas : C:/, D:/ etc.
 - Une racine commune root : /
 - Les Périphériques sont des sous-répertoires : /mnt/cdrom
- Compte utilisateur :
 - Espace disque privé : /private/student/5/25/2363415 ou ~/2363415
 - Appartient à un groupe
- Répertoires et fichiers : Droits d'accès
- Système d'interface : Shell



A screenshot of a terminal window titled "akritika — bash — 80x24". The window shows the following text:

```
Last login: Sun Sep  2 16:33:38 on console  
aisopos:~ akritika$
```

Below the prompt, there is a large, dark gray rectangular area. In the top right corner of this area, the word "Shell" is written in a bold, black font.

- Interface en ligne de commande (CLI)
 - Entrée : Les commandes que vous souhaitez exécuter (utilisation du clavier)
 - Sortie : Le Shell transmet les commandes au système d'exploitation, qui les exécute et fournit le résultat (généralement à l'écran)
 - Scripts : Combinaison de commandes
 - Variables

- Interaction avec le Shell : Émulateurs de terminal
 - Konsole, Xterm, Gnome-terminal...
- Plusieurs shells sont disponibles
 - `sh(shell Bourne)`, `bash(shell Bourne)`, `csch`, `zsh`, `ksh`...
- Chaque utilisateur a un Shell pré-sélectionné
 - Sélection dans le fichier : </etc/passwd>
 - Commande pour changer le Shell : `chsh`

- Répertoire personnel (\$HOME)
 - Chaque utilisateur a un répertoire personnel lié à son nom
 - Nom : 2363415
 - Répertoire Home : /private/student/5/25/2363415 ou ~/2363415
- Répertoire de travail (\$PWD)
 - Répertoire courant
 - Lorsque vous êtes connecté : Répertoire de travail = Répertoire personnel
- Répertoire courant : .
- Répertoire parent : ..
- Répertoire Root : /
- Répertoire personnel de l'utilisateur : ~

■ Absolu

- From root directory / : [/private/student/5/25/2363415](#)

■ Relatif

- À partir du répertoire de travail :
 - ✗ Supposons le répertoire de travail suivant :
[/private/student/5/25/2363415](#)
 - ✗ Vous pouvez accéder à un fichier dans un autre répertoire [/private/student/5/25/2363414](#) par :
[../2363414](#)

- Caractères génériques spéciaux qui peuvent être utilisés en combinaison avec des commandes
- Un seul et unique caractère : ?
 - Ex : `ls /etc/rc????`
- De 0 à plusieurs caractères : *
 - Ex: `ls /etc/rc*`
- Caractères spécifiques : [...]
 - Ex : `ls [abc]oo.c`
`aoo.c, boo.c, coo.c`

- Préserver la valeur littérale du caractère suivant : `\`
- Préserver la valeur littérale d'une séquence de caractères :
`'...'` (guillemets simples)
- Comme précédemment, sauf pour les caractères `$` et ``` :
`"..."` (guillemets doubles)
- Remplacement de commande : Exécution de la commande et remplacement par ce qu'elle a produit en sortie
``command`` (guillemets inversés)
`$ (command)`

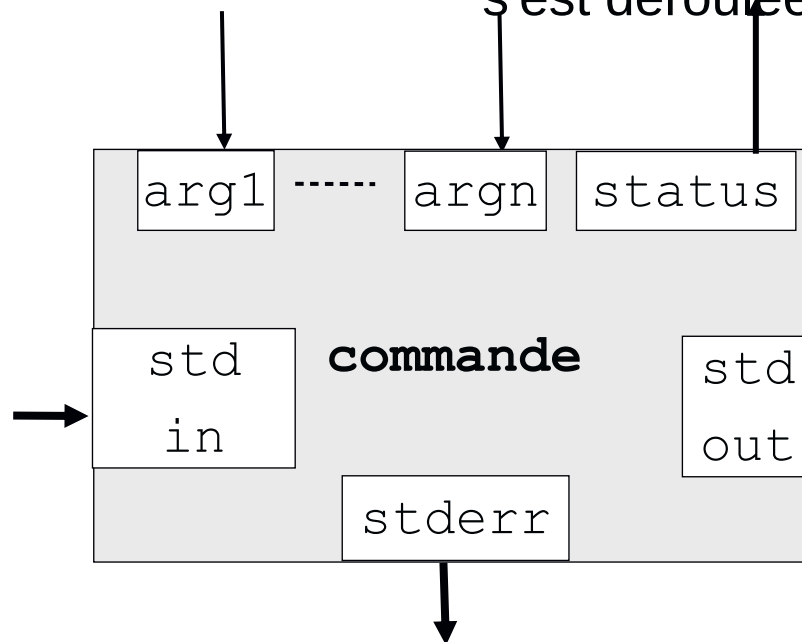
- Touche Tab :
 - Une fois : remplit automatiquement le mot que vous avez commencé à taper, s'il existe
 - Deux fois : Affiche tous les mots existants qui commencent par les lettres que vous avez tapées, s'ils existent
- Commande **clear** : Nettoie l'écran dans le Shell
- Touche flèche vers le haut : Affiche la commande précédente
- Commande **history** : Affiche toutes les commandes
- **!!** : Exécute la commande précédente
- N'utilisez pas de caractères spéciaux dans les noms de fichiers, y compris l'espace et le tiret (-).
- Définissez la langue par défaut en ANGLAIS, la traduction en français n'est pas complète... (LANG=EN)"

Structure d'une commande

commande arg1 arg2 ... argn

Les arguments d'une commande sont les paramètres passés lors de son appel.

Le statut de sortie est la valeur renvoyée après l'exécution de la commande pour informer si l'exécution s'est déroulée normalement ou non.



L'entrée standard est lue depuis le clavier –

Le numéro 0 est `stdin`

- Raccourci : -

La sortie standard est imprimée à l'écran

- Le numéro 1 `stdout`

Par défaut, les erreurs standard sont imprimées à l'écran et sont utilisées par les commandes pour afficher les messages d'erreurs

- (Le numéro 2 est `stderr`)

- Gestion des répertoires
 - Gestion des fichiers
 - Redirections
 - Gestion des processus
 - Programmation Shell : Scripts
-
- Ce cours NE fournit PAS une liste exhaustive des commandes et une description complète des commandes
→ Utilisez :
 - La documentation
 - La commande **man** ou **info**
 - La commande **apropos**

- Afficher le chemin complet du répertoire de travail : **pwd**
- Changer de répertoire : **cd**
 - **cd** ou **cd ~** : Accéder au répertoire personnel
 - **cd path** : Accéder au répertoire indiqué par **path**
- Création/Suppression de répertoires
 - **mkdir dirname** : Créer un repertoire nommé **dirname**
 - **rmdir dirname** : Supprimer le répertoire nommé **dirname** (vide)
 - **rm -r dirname** : Supprime le répertoire nommé **dirname** de manière récursive (y compris les sous-répertoires) (non vide)
 - **PAS DE CORBEILLE !**
- Renommer : **mv dirsource dirdest**
- Copie hiérarchique (sous-répertoires) : **cp -r dirsource dirdest**

- **ls** : Liste les fichiers dans le répertoire de travail
- **ls dirname** : Afficher le contenu du répertoire **dirname**

```
angie@achilleas:~/Entertainment$ ls
movies Music photos
```

- Certains paramètres passés en tant qu'arguments :

- **-l**: Utiliser un format de liste longue (détails)

```
angie@achilleas:~/Entertainment$ ls -l
total 12
drwxrwxr-x 8 angie angie 4096 Φεβ  1 2015 movies
drwxr-xr-x 3 angie angie 4096 Φεβ  1 2015 Music
drwxrwxr-x 5 angie angie 4096 Οκτ 20 2015 photos
```

- **-a** : Ne pas ignorer les entrées commençant par . (dossiers/fichiers cachés)

```
angie@achilleas:~/Entertainment$ ls -al
total 28
drwxrwxr-x  5 angie angie  4096 Δεκ  4 2014 .
drwxr-xr-x 48 angie angie 12288 Μάρ 30 08:53 ..
drwxrwxr-x  8 angie angie  4096 Φεβ  1 2015 movies
drwxr-xr-x  3 angie angie  4096 Φεβ  1 2015 Music
drwxrwxr-x  5 angie angie  4096 Οκτ 20 2015 photos
```


- Créer un fichier en :
 - mettant à jour l'heure d'accès et de modification : **touch filename**
- Supprimer un fichier : **rm filename**
- Renommer : **mv filesource filedest**
- Copier :
 - **cp filesource filedest**
 - **cp filesource dirdest**
- Supprimer la partie de suffixe qui n'est pas un répertoire d'un nom de fichier : **dirname**
- Supprimer le suffixe de la partie répertoire d'un nom de fichier : **basename**
- Compter le nombre de mots dans un fichier : **wc -w filename**

- Afficher le contenu d'un fichier :
 - **more filename** : Affiche page par page le contenu du fichier **filename** sur la sortie standard (`stdout`) (Q : Quitter la commande 'more')
 - **less filename** : Similaire à 'more', mais il permet de naviguer vers le haut et vers le bas entre les pages (Q : Quitter la commande 'less').
 - **head -n 42 filename** : Afficher les 42 1ère ligne de **filename**
 - **tail -n 42 filename** : Afficher les 42 dernières ligne de **filename**

- Concaténer des fichiers (créer et afficher des fichiers) :
 - `cat filename1 ... filenameN` (Imprimer le contenu des fichiers sur la sortie standard `stdout`)
 - `Cat` : Afficher sur la sortie standard (`stdout`) tout ce qui est saisi sur l'entrée standard (`stdin`) (`cntrl+D` indique la fin de fichier `EOF`)
- Différence entre deux fichiers :
 - `diff filename1 filename2`

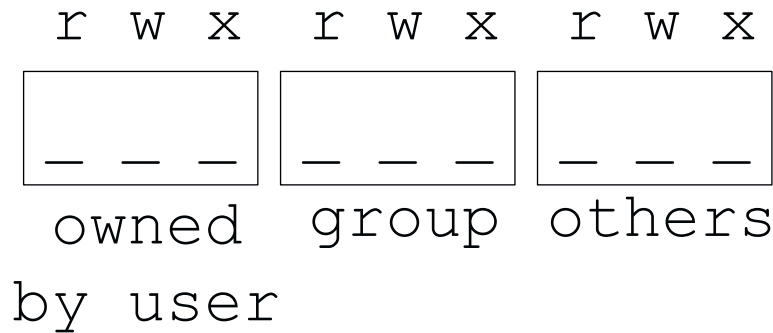
- Associer plusieurs noms de fichiers à un seul fichier
- En dur : **ln filename1 filename2**
 - Le nouveau nom de fichier (filename2) pour accéder à un fichier existant (filename1)
 - Impossible de lier des répertoires
 - Impossible à travers les limites du système de fichiers
 - La source du lien est supprimée : Le lien fait toujours référence à la source
- Symbolique : **ln -s filename1 filename2**
 - Un chemin symbolique indiquant l'emplacement abstrait d'un autre fichier
 - Peut lier des répertoires
 - Peut traverser les limites du système de fichiers
 - La source du lien est supprimée : Le lien n'est pas mis à jour

QCM !

Commandes Linux

- A. Quelle commande devez-vous utiliser pour accéder au répertoire parent ?
- 1) `cd ~`
 - 2) `cd -`
 - 3) `cd ..`
 - 4) `cd /up`
- B. Comment renommez-vous un fichier de `oldname` à `newname` ?
- 1) `mv oldname newname`
 - 2) `cp oldname newname`
 - 3) `move oldname newname`
 - 4) `rn oldname newname`

- c. Comment copiez-vous l'ensemble de la structure du répertoire de project à project_backup ?
 - 1) `cp -d project project_backup`
 - 2) `cp -s project project_backup`
 - 3) `cp -e project project_backup`
 - 4) `cp -r project project_backup`
- D. Supposons que je sois dans le répertoire /etc, après avoir exécuté "cd ." (un seul point), où suis-je maintenant ?
 - 1) /etc
 - 2) /
 - 3) /home
 - 4) ~/user



■ Changer permissions : **chmod**

`chmod [who] op permissions [,op permissions ]* name`

■ Who :

- Owned by user (u)
- Group (g)
- Others (o)
- Nothing (all)

■ Op :

- Add (+)
- Remove (-)
- Define (=)

■ Permissions:

- Read or List (r)
- Write (w)
- Execute or Access(x)

r	w	x	r	w	x	r	w	x
— — —			— — —			— — —		
owned			group			others		
by user								

- Les autorisations peuvent être attribuées avec un code numérique par champ
- Dans un champ, vous mettez 1 ou 0 dans chaque colonne pour donner ou ne pas donner l'autorisation correspondante..

1	1	1	1	1	0	1	0	0
owned			group			others		
by user								
↓			↓			↓		
7			6			4		

chmod 764 filename

QCM

- A. Que fait la commande : `chmod o-rwx f`
 - 1. Accorde toutes les autorisations au propriétaire du fichier `f`.
 - 2. Supprime toutes les autorisations des autres utilisateurs du fichier `f`.
 - 3. Supprime toutes les autorisations du propriétaire du fichier `f`.
 - 4. Erreur de syntaxe

- B. Pour la commande : `chmod +x file`
 - 1. Accorde toutes les autorisations au propriétaire du fichier f.
 - 2. Supprime toutes les autorisations des autres utilisateurs du fichier f.
 - 3. Supprime toutes les autorisations du propriétaire du fichier f.
 - 4. Erreur de syntaxe

- B. Que fait la commande : `chmod g=rx file`
1. Donne toutes les autorisations au groupe d'utilisateurs.
 2. Donne les autorisations de lecture et d'exécution au groupe d'utilisateurs.
 3. Supprime les autorisations de lecture et d'exécution du groupe d'utilisateurs.
 4. Erreur de syntaxe

- Rechercher des fichiers en fonction de critères (afficher le chemin) :

find répertoire [critères]

- La recherche est effectuée dans le répertoire (de manière récursive dans les sous-répertoires)
- Appliquer une commande aux fichiers trouvés : option **-exec cmd { } \;**
 - ✗ Appliquer la commande cmd à chaque fichier trouvé (elle remplace { })
 - ✗ Fin de la commande : \;

- Quelques critères de recherche :
 - **-name filename** : basé sur le nom de fichier (**-iname** : ignorer la case)
 - **-user username** : basé sur l'utilisateur propriétaire du fichier
 - **-size number [c | k | M]** : basé sur la taille [o, Ko, Mo]
 - **-type [d | f | l]** : basé sur le type [répertoire, fichier, lien]
 - **-mtime x** : basé sur la date de modification (moins de x jours)
 -
- Les critères peuvent être combinés : **-o -a -not**

- `find /usr -name "*.c"`
- `find /usr -user angeliki`
- `find . -not \( -name "*.obj" -o -name "*.map" \)`
- `find . \( -user angeliki -a -name "*.c" \)`
- `find . -name "*.java" -exec lpr {} \;`

QCM

- A. Pour trouver les fichiers .c à l'intérieur du répertoire /usr :
1. `find /usr -name "*.c"`
 2. `find usr -name "*.c"`
 3. `find /usr -name *.c`
 4. `find usr -name *.c`

- B. Pour trouver les fichiers .c à l'intérieur du répertoire de travail (working directory) dont le propriétaire est "angeliki" :
1. `find . \( -owner angeliki -a -name "*.c" \)`
 2. `find . \( -user angeliki -a -name "*.c" \)`
 3. `find \( -owner angeliki -a -name "*.c" \)`
 4. `find \( -user angeliki -a -name "*.c" \)`

- Afficher toutes les lignes des fichiers du répertoire qui contiennent un mot : **grep string directory**
 - La chaîne peut être exprimée sous forme d'expression
 - ✗ N'importe quel caractère : .
 - ✗ Répétition du motif : *
 - ✗ Intervalle de caractères : [**x-y**]
 - Retourner la valeur : 0 not found, else !=0
- Exemples :
 - `grep -r test ./`
 - `grep "m.*t" filename`
 - `grep "toto[1-3]" filename`

- Redirection de l'entrée standard (**stdin**) :
 - Utiliser le contenu d'un **fichier** comme entrée pour une commande (au lieu du clavier)
 - ✗ `command < filename`
- Redirection de sortie standard (**stdout**)
 - Imprimer la sortie d'une commande dans un fichier (au lieu de l'écran)
 - ✗ `command > filename` (Écraser le contenu ancien du fichier)
 - ✗ `command >> filename` (ajouter au contenu du fichier)
- Pipe : rediriger la sortie standard vers l'entrée standard des **commandes**
 - Utiliser la sortie d'une commande comme entrée pour une autre commande
 - Un sous-shell est créé pour exécuter la commande_2
 - ✗ `command_1 | command_2`

Redirections: Examples

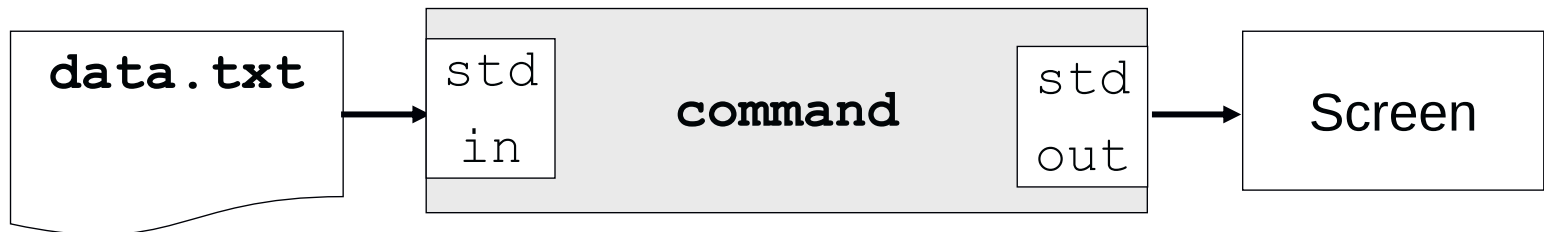
■ `command`



■ `command > res.txt`



■ `command < data.txt`

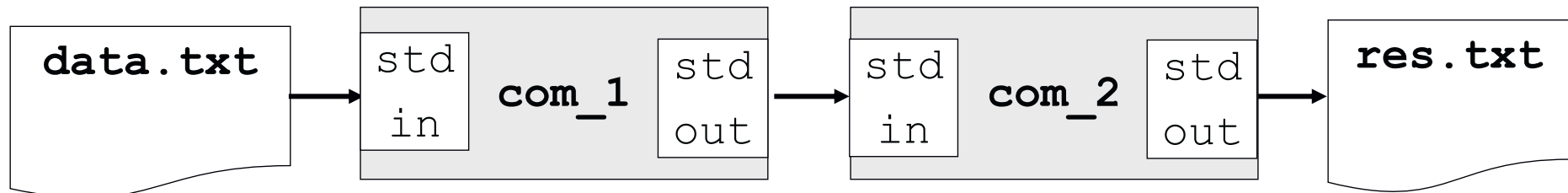


Redirections: Examples

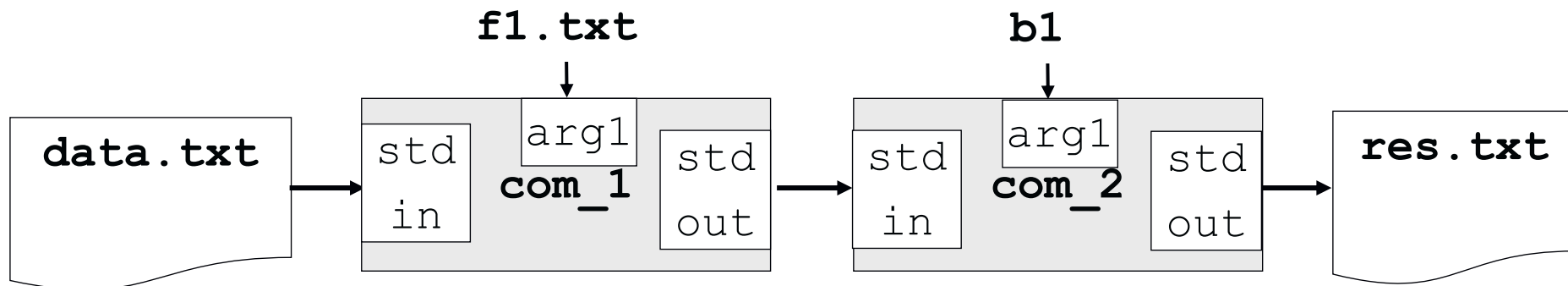
- `command < data.txt > res.txt`



- `com_1 < data.txt | com_2 > res.txt`



- `com_1 f1.txt < data.txt | com_2 b1 > res.txt`



QCM

A. Que fait la commande : `cat < file1 >> file2 | file3`

1. Le contenu de `file1` est ajouté à `file2` et stocké dans `file3`
2. Le contenu de `file1` est ajouté à `file2` et `file3` est ignoré
3. `file2` et `file3` ont le même contenu
4. Erreur de syntaxe

B. Laquelle des commandes suivantes redirige le message d'erreur vers le fichier "errorfile" :

- 1- `cat command errorfile`
- 2- `cat command > errorfile`
- 3- `cat errorfile > command`
- 4- `cat command 2> errorfile`

C- Que fait la commande : `cat < file1 >> file2 | file3`

- 1- Le contenu de `file1` est ajouté à `file2` et stocké dans `file3`
- 2- Le contenu de `file1` est ajouté à `file2` et `file3` est ignoré
- 3- `file2` et `file3` ont le même contenu
- 4- Erreur de syntaxe

■ Interne

- Implémenté dans le shell
- Ils affectent l'état interne du shell (ex. `cd`)

■ Externe

- Implémenté en tant que code séparé
- Le shell interprète la commande comme une demande de chargement et d'exécution du programme qui implémente la commande
- Stocké dans le `/bin`

Commande	Description	Exemple
cd	Changer de répertoire	cd ..
declare	Définir une variable	declare myvar
echo	Afficher sur la sortie standard (stdout)	echo hello
exec	Remplacer bash par un autre processus	exec ls
exit	Quitter bash	exit
export	Définir variable globale	export myvar=1
history	Afficher l'historique des commandes	history
kill	Envoyer un signal à un processus	kill 1121
let	Calculer une opération arithmétique	let myvar=3+5

Commande	Description	Exemple
<code>local</code>	Définir une variable locale	<code>local myvar=5</code>
<code>pwd</code>	Afficher le repertoire de travail	<code>pwd</code>
<code>read</code>	Lire depuis l'entrée standard (<code>stdin</code>)	<code>read myvar</code>
<code>readonly</code>	Verrouiller une variable	<code>readonly myvar</code>
<code>return</code>	Fin d'une fonction et retourne une valeur	<code>return 1</code>
<code>set</code>	Afficher les variables	<code>set</code>
<code>shift</code>	Déplacer des paramètres	<code>shift 2</code>
<code>test</code>	Tester une condition	<code>test -d temp</code>
<code>trap</code>	Suivre un signal	<code>trap "echo signal" 3</code>

- Processus : L'instance en cours d'exécution d'un programme, par exemple :
 - Exécution d'une commande
 - Exécution de votre programme

- Exécution en premier plan : Processus interactifs
 - Connecté au Shell
 - L'utilisateur peut envoyer des entrées

- Exécution en arrière-plan : Processus non interactifs ou automatisés
 - Non connecté à un terminal
 - L'utilisateur ne peut pas envoyer d'entrées tant que le processus n'est pas déplacé en premier plan

- Exécuter en arrière-plan : **&**:

- Ex: `gedit &`

- De premier plan à arrière-plan :

- Ex: `gedit`

Press CTRL + Z (met le processus dans un état d'attente)
`bg`

- D'arrière plan au premier plan :

- Ex: `fg %jobid`

(jobid n'est pas un PID. C'est un numéro unique attribué par le Shell, par exemple lorsque le processus passe en arrière-plan.)

- Processus : L'instance en cours d'exécution d'un programme, par exemple . :
 - Exécution d'une commande
 - Exécution de votre programme
- Exécution séquentielle ou en série : ;
 - Toutes les commandes sont exécutées
 - Nouvelle invite de commande lorsque la dernière commande se termine
 - Ex : `who | sort ; date`
- Exécution parallèle ou simultanée : &
 - Toutes les commandes sont exécutées
 - Une nouvelle invite de commande est immédiatement disponible (exécution en arrière-plan)
 - Ex : `pr junk | lpr &`

- Afficher le catalogue des processus en cours d'exécution : **ps**
 - **-a** : Sélectionner tous les processus associés à un terminal.
 - **PID** : Identifiant unique du processus
 - **TTY** : Console dans laquelle le processus est créé
 - **TIME** : Temps total d'exécution (en secondes)
 - **CMD** : Commande exécutée

```
angie@achilleas:~/Downloads$ ps
  PID TTY          TIME CMD
 12190 pts/1        00:00:00 bash
 14414 pts/1        00:00:00 vi
 14447 pts/1        00:00:00 evince
 14454 pts/1        00:00:00 ps
```

- Terminer un processus en cours d'exécution
 - Méthode globale :
 - ✗ **kill PID**
 - Exécution en premier plan dans l'interpréteur de commandes actuel (à travers les SIGNAUX)
 - ✗ **CTRL + C** (Interruption)
 - ✗ **CTRL + D** (**EOF**) => **End of File**
 - ✗ **CTRL + ** (Quitter)
- Mettre un processus en pause
 - **CTRL + Z**
- Résumé
 - Foreground 'Premier plan' (**fg**)
 - Background 'arrière-plan' (**bg**)

- Valeurs initiales : définies dans le fichier de configuration système et utilisateur
- Utiliser la majuscule par convention
- Définition des variables en fonction du type (bash shell)
 - variables String : **MYVAR="value"** (**PAS D'ESPACES AUTOUR DE =**)
 - variables Integer : **declare -i MYVAR**
 - variables Constant : **readonly DATA=value**
 - variables Array (Tableau) : **declare -a MYARRAY**
MYARRAY[0]="one" ; MYARRAY[1]=5
- Utilisation des variables : **\$**
 - Ex. **\$MYVAR**
- Lifetime of variables :
 - Fin de session – terminaison du shell
 - Supprimé par l'utilisateur : **unset VARNAME**

- Variables Globales : Variable publique visible par tous les sous-Shell
 - Définir l'attitude d'exportation pour la variable : **export**
 - Afficher les variables globales : **env**, **printenv**
- variables Locales : Visible par le shell courant
 - Afficher les variables globales : **set**

Variables Shell : Exemple de portée

```
angie@achilleas:~$ myvar="hello"
angie@achilleas:~$ set | grep myvar
myvar=hello
angie@achilleas:~$ echo "$$"
12190
angie@achilleas:~$ bash
angie@achilleas:~$ echo "$$"
15752
angie@achilleas:~$ set | grep myvar
angie@achilleas:~$ env | grep myvar
angie@achilleas:~$ exit
exit
angie@achilleas:~$ export myvar="hello"
angie@achilleas:~$ bash
angie@achilleas:~$ echo "$$"
15825
angie@achilleas:~$ set | grep myvar
myvar=hello
angie@achilleas:~$ █
```

Variable	Description
USER	Nom du compte utilisateur
HOME	Répertoire personnel de l'utilisateur
TERM	Type de terminal
SHELL	Nom du shell
PATH	Liste de répertoires contenant des commandes exécutables
MANPATH	Liste de répertoires contenant des informations d'aide pour les commandes
PWD	Répertoire courant
OLDPWD	Répertoire courant précédent
HOSTNAME	Nom du système

- Programme composé de commandes :
 - Utilisation d'un editeur :
 - ✗ **vi (vim), emacs, sublime, gedit, ...**
 - La première ligne définit la coquille à utiliser :
 - ✗ **#!/bin/bash** (Pour shell for bash)
 - Écrire les **commandes du programme**
 - Les commandes peuvent être séparées par :
 - ✗ Une nouvelle ligne
 - ✗ Point-virgule : **;**
 - Séparer une commande sur 2 lignes ou plus : **** (anti slash)
 - Les commentaires : **#**
 - Enregistre les fichiers avec l'extension **.sh**
- Les lignes du script (commandes) sont interprétées et NON compilées.

- Pas besoin d'autorisations :

```
bash path_to_script/myscript.sh
```

- La commande **bash** est exécuté
- **./script.sh** est l'argument en ligne de commande.

- Fournir des permissions d'exécution et exécuter :

```
chmod +x myscript.sh
```

```
./myscript.sh
```

- Le shell se clone lui-même et utilise un appel système (par exemple, `execve`) pour que le système d'exploitation exécute le fichier dans le processus cloné.
- L'OS vérifie les permissions du fichier
- Le chargeur de programme examine le fichier pour déterminer comment l'exécute

(#! pour les scripts, puis quel interpréteur à utiliser : **/bin/bash**).

- Nouveau processus : Attention aux variables et aux fonctions

- Des arguments peuvent être transmis lors de l'exécution du script :
 - `./myscript.sh arg1 ... argN`
- Les arguments sont perçus comme des variables spécifiques dans le code du script :
 - `$0`: Nom du script
 - `$1`: 1^{er} argument
 - `$N`: nième argument
 - `$#`: Nombre d'arguments
 - `$*`: Tous les arguments transmis au script, en tant que chaîne de caractères
 - `$@`: Se comporte comme `$*` sauf que lorsque les arguments sont entre guillemets, ils sont correctement découpés s'il y a des espaces en eux.
- La longueur maximale des arguments est définie par le système d'exploitation et est mesurée en kilo octets.
 - `getconf ARG_MAX`

■ Initialisation : `nomvar=expression`

- **ATTENTION! Pas d'espace autour de =**

- `expression`:

- ✗ Valeur

- ✗ Référence à une variable

- ✗ ``commande`` (remplacé par le résultat pour le `stdout`)

■ Utiliser : `$nomvar`

- Afficher le repertoire courant

```
y=`pwd`
```

```
echo $y
```

- Afficher les deux premiers argument d'un script

```
y="$1 and $2"
```

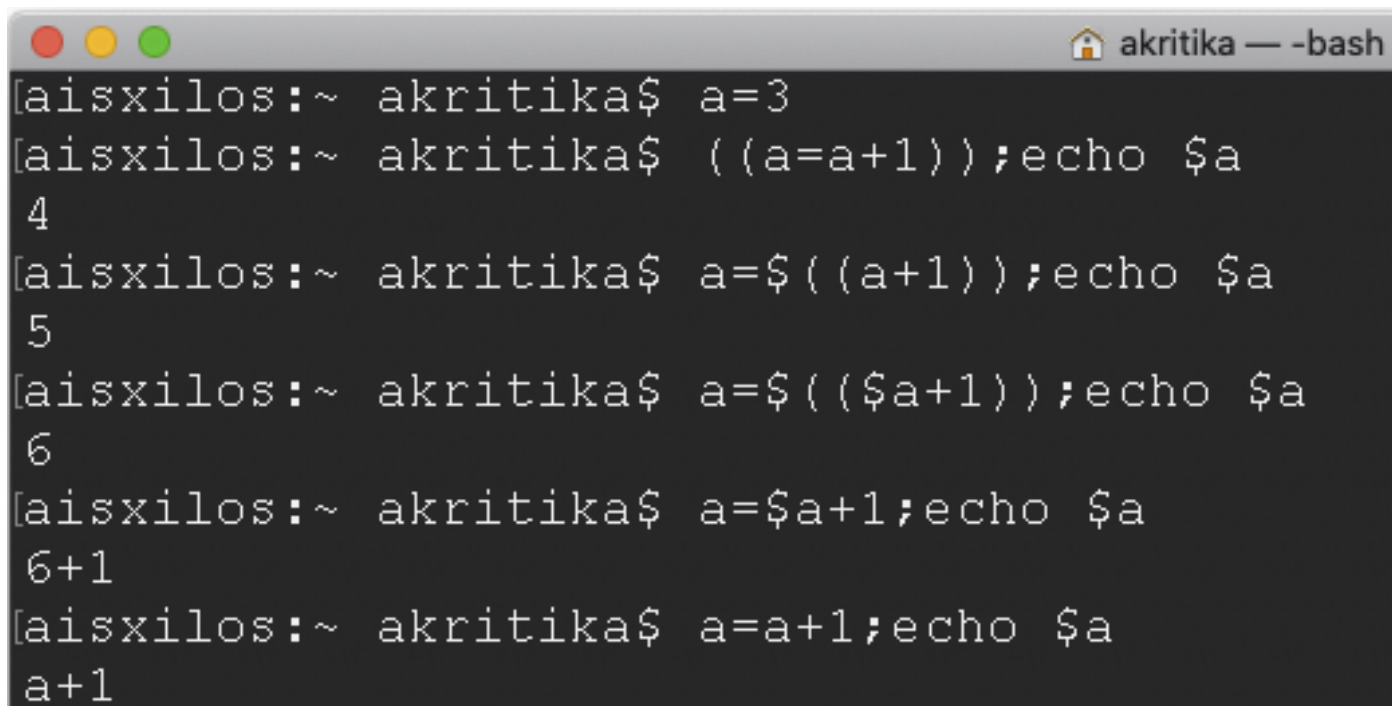
```
echo $y
```

- Nous pouvons demander à l'utilisateur de fournir certaine(s) valeur(s) aux variables
 - `read var-name1 var-name2 ... var-nameN`
 - Lit une ligne depuis l'entrée standard (stdin) et affecte la première valeur à var-name1, la deuxième valeur à var-name2, etc.
 - Retourne 0, si la lecture a réussi
- Nous pouvons afficher un message à l'utilisateur :
 - `read -p "Enter value:" var`
- Si la variable n'est pas spécifiée, alors la valeur par défaut est utilisée :
 - `$REPLY`

- Affichage de base vers `stdout` : **echo**
 - Strings
 - Variables
 - Caractères spéciaux
 - Peut avoir plusieurs lignes

```
angie@achilleas:~/Entertainment$ echo hello
hello
angie@achilleas:~/Entertainment$ echo *
movies Music photos
angie@achilleas:~/Entertainment$ echo print '@' "user"
print @ user
angie@achilleas:~/Entertainment$ echo 'hell
> o there'
hell
o there
```

- Pas besoin de définir les variables en tant que des entiers
- Operations arithmetique avec integers
 - `$ ( (...) )` : Portabilité entre les interpréteurs de commandes
 - `((...))`



A terminal window titled "akritika — -bash" showing a series of commands and their outputs. The prompt is [aisxilos:~ akritika\$]. The commands and outputs are as follows:

```
[aisxilos:~ akritika$ a=3
[aisxilos:~ akritika$ ((a=a+1));echo $a
4
[aisxilos:~ akritika$ a=$((a+1));echo $a
5
[aisxilos:~ akritika$ a=$(( $a+1 ));echo $a
6
[aisxilos:~ akritika$ a=$a+1;echo $a
6+1
[aisxilos:~ akritika$ a=a+1;echo $a
a+1
```

- Opérations Arithmétiques avec integers
 - **let** (sans espaces car c'est une initialisation)
 - **expr** (avec des espaces dans les arguments)

```
[aisxilos:~ akritika$ a=3
[aisxilos:~ akritika$ let a=a+1; echo $a
4
[aisxilos:~ akritika$ let a++; echo $a
5
[aisxilos:~ akritika$ a=`expr $a + 1`; echo $a
6
[aisxilos:~ akritika$ a=`expr $a+1`; echo $a
6+1
[aisxilos:~ akritika$
```


- Une expression conditionnelle est :
 - un ensemble de commandes qui sont exécutées
- La commande de test se termine avec le statut de sortie déterminé par **expression_condition**
 - `test condition_expression`
 - `[condition_expression]`
 - `[[condition_expression ]]`

**ESPACES SONT
REQUIS !!**

Similaire aux crochets simples, mais plus puissant

- `(( arithmetic condition_expression ))`
- `( command )`

Condition expressions: test operators

Opérateur	Description
<code>-gt</code> or <code>></code>	Supérieur à (<code>-gt</code> pour les entiers et <code>></code> pour les chaînes, " <code><</code> " doit être échappé (utilisez <code>\</code>) à l'intérieur d'une construction <code>[]</code>)
<code>-ge</code>	Supérieur ou égal (combinez <code>!</code> et <code><</code> pour les chaînes)
<code>-lt</code> or <code><</code>	Inférieur à (<code>-lt</code> pour les entiers et <code><</code> pour les chaînes)
<code>-le</code>	Inférieur ou égal (combinez <code>!</code> et <code>></code> pour les chaînes)
<code>-eq</code> or <code>=</code>	Égal (<code>-eq</code> pour les entiers et <code>=</code> pour les chaînes)
<code>-ne</code> or <code>!=</code>	Non égal (<code>-ne</code> pour les entiers et <code>!=</code> pour les chaînes)
<code>-n str</code>	Chaîne de taille non nulle
<code>-z str</code>	Chaîne de taille nulle
<code>-d dirname</code>	<code>dirname</code> est un répertoire
<code>-s filename</code>	<code>filename</code> la taille n'est pas nulle
<code>-f filename</code>	<code>filename</code> existe
<code>-r filename</code>	<code>filename</code> Existe et nous avons accès en lecture
<code>-w filename</code>	<code>filename</code> Existe et nous avons accès en écriture
<code>-x filename</code>	<code>filename</code> Existe et nous avons un accès en exécution

■ Syntaxe

\$?

ATTENTION !!

■ Fonction

- Si la commande se termine correctement (**TRUE**) : Se termine avec la **valeur 0** (exit 0)
- S'il y a eu une erreur (**FALSE**) : se termine avec la valeur **1** (exit 1)

```
$ true ; echo $?  
$ 0  
$ false ; echo $?  
$ 1
```

■ Commandes qui ne font rien d'autre que renvoyer un statut de sortie de :

- **true**: Renvoie un statut de sortie de zéro
- **false**: Renvoie un statut de sortie de un

QCM

```
#!/bin/bash  
a=10  
b=$(( $a<100 ))  
echo $b  
exit 0
```

A. Quelle est la sortie de ce script ?

1. 0
2. 1

B. Qu'affiche echo \$? après l'exécution de ce script ?

1. 0
2. 1

- Syntaxe (AND) : `-a` ou `&&`
`commande1 && commande2`
- Fonction :
 - `commande2` est exécutée si, et seulement si, `commande1` renvoie un statut de sortie de zéro (TRUE).
- Syntaxe (OR) : `-o` ou `||`
`commande1 || commande2`
- Fonction:
 - `commande2` est exécutée si, et seulement si, `commande1` renvoie un statut de sortie non nul (FALSE).
- Le statut de sortie des listes AND et OR est le statut de sortie de la **dernière commande** exécutée dans la liste
- Syntax (NOT): `!`

(QCM)

A. Quelle est l'instruction de comparaison correcte en script shell ?

- 1- if \$x -gt \$y
- 2- if [\$x -gt \$y]
- 3- if (\$x -gt \$y)
- 4- Aucune

B. Quelle est la sortie de `true || echo "echo executed" ;`
`echo $?`

1. 0
2. echo executed 0
3. echo executed 1
4. 1

C. Quelle est la sortie de `false || echo "echo executed" ;`
`echo $?`

1. 0
2. echo executed 0
3. echo executed 1
4. 1

D. Quelle est la sortie de `true && echo "echo executed" ;`
`echo $?`

1. 0
2. echo executed 0
3. echo executed 1
4. 1

E. Quelle est la sortie de `false && echo "echo executed" ;`
`echo $?`

1. 0
2. echo executed 0
3. echo executed 1
4. 1

```
$ true || echo "echo executed"
$ false || echo "echo executed"
echo executed
$ true && echo "echo executed"
echo executed
$ false && echo "echo executed"
$
```

■ Syntaxe :

```
if [ condition_expression_1 ] ; then
    1st set of commands
elif [ condition_expression_2 ]
then
    2nd set of commands
else
    3rd set of commands
fi
```

**SPACES for [] ARE
REQUIRED !!**

■ Fonction :

- Le `if` vérifie si la `condition_expression_1` retourne TRUE
 - ✗ Le premier ensemble de commandes est exécuté
- Else, si la `condition_expression_2` retourne TRUE
 - ✗ Le second ensemble de commandes est exécuté
- Sinon, le troisième ensemble de commandes est exécuté

```
#!/bin/bash
read -p "Enter a filename: " filename
if [ ! -w "$filename" ]; then
    echo "File $filename is not writable"
    exit 1
elif [ ! -r "$filename" ]; then
    echo "File $filename is not readable"
    exit 1
else
    echo "File $filename is writable and readable"
fi
```

Conditions: if : Examples

```
#!/bin/bash
TMPFILE="diff.out"
diff $1 $2 > $TMPFILE
if [ ! -s "$TMPFILE" ]; then
    echo "Files are the same"
else
    more $TMPFILE
fi
if [ -f "$TMPFILE" ]; then
    rm $TMPFILE
fi
```


■ Syntaxe :

```
case $variable in
    pattern 1 )
        1st set of commands ;;
    pattern 2 | pattern 3 )
        2nd set of commands ;;
    *)
        Nth set of commands ;;
esac
```

■ Fonction :

- Si la valeur de la **variable** est **pattern 1**, le premier ensemble d'instructions est exécuté, si c'est **pattern 2**, le deuxième ensemble d'instruction est exécuté, etc.
- Sinon (*) → N^{ième} ensemble d'instruction

Conditions : case: Exemple

```
#!/bin/bash
read -p "Enter a command: " command
case $command in
all | ALL )
    echo "Display all files..."
    ls -la ;;
list | LIST)
    echo "Display all non-hidden
files..."
    ls -ll ;;
*)
    echo "Invalid choice"
esac
```

■ Syntaxe :

```
for variable in list_of_values  
do  
    ensemble d'instructions  
done
```

■ Fonction

- À chaque itération, la variable prend une valeur à partir de **liste_de_valeurs**
 - ✗ L'ensemble d'instructions est exécuté pour cette valeur
- **liste_de_valeurs** peut être :
 - ✗ Strings : **L3info L3miage M1info**
 - ✗ Numbers : **counting: {1..10..2}**
enumerating : 1 3 5
 - ✗ Arguments de ligne de commande : **\$***
 - ✗ Nom de fichier : **filename1 filename2**
 - ✗ Commande de sortie : **\$(ls /tmp/*)**

```
#!/bin/bash
for file in f1 f2 f3
do
    cp $file $1/$file
done
```

```
#!/bin/bash
for i in 6 3 1 2
do
    echo $i
done | sort -n
```

- Syntaxe :

```
while
    [ condition_expression ]
do
    ensemble d'instructions
done
```

- Fonction

- **Tant que l'expression de condition renvoie TRUE**
 - ✗ L'ensemble de commande est exécuté
- Sinon, l'iteration se termine

```
#!/bin/bash
i=1
while [ $i -lt 10 ]
do
    echo $i
    ((i++))
done
```

```
#!/bin/bash
while true
do
    echo "alive..."
    sleep 3s
done
```

■ Syntaxe :

```
until
    [ condition_expression ]
do
    ensemble d'instructions
done
```

■ Fonction

- Exécute le 1er ensemble d'instruction
- Tant que les commandes de condition_expression NE sont PAS vraies (tant que condition_expression est FAUSSE)
 - ✗ L'ensemble d'instructions est exécuté
- Sinon fin de l'itération

```
#!/bin/bash
i=1
until [ $i -ge 10 ]
do
    echo $i
    ((i++))
done
```


- Arrêter avec l'itération actuelle de la boucle : **break**
- Continuer avec la prochaine itération : **continue**
- Terminer le script : **exit**

```
#!/bin/bash
for x in 1 2 3 4 5 6 7 8 9
do
    echo -n "x is $x. "
    if [ "$x" -eq "4" ]; then
        continue
    fi
    echo "Most numbers get here, except 4."
    if [ "$x" -eq "7" ]; then
        break
    fi
done
echo "We have finished the loop now."
echo "At the end, x is now $x."
```

```
x is 1. Most numbers get here, except 4.  
x is 2. Most numbers get here, except 4.  
x is 3. Most numbers get here, except 4.  
x is 4.  
x is 5. Most numbers get here, except 4.  
x is 6. Most numbers get here, except 4.  
x is 7. Most numbers get here, except 4.  
We have finished the loop now.  
At the end, x is now 7.
```

■ Syntaxe

```
function name[ () ]  
{  
    list des commandes;  
[return]}
```

■ Fonction

- Les fonctions doivent être définies au début du script
- Il se peut qu'elles aient ou non des arguments
- Les arguments et les valeurs de retour peuvent être de n'importe quel type
- Les variables définies à l'intérieur de la fonction sont GLOBALES. Elles doivent être définies LOCALES si nous voulons réduire leur portée.

```
#!/bin/bash
outside="a global variable"
function mine() {
    local inside="this is local"
    echo $outside
    echo $inside
    outside="a global with new value"
}
echo $outside
mine
echo $outside
echo $inside
```

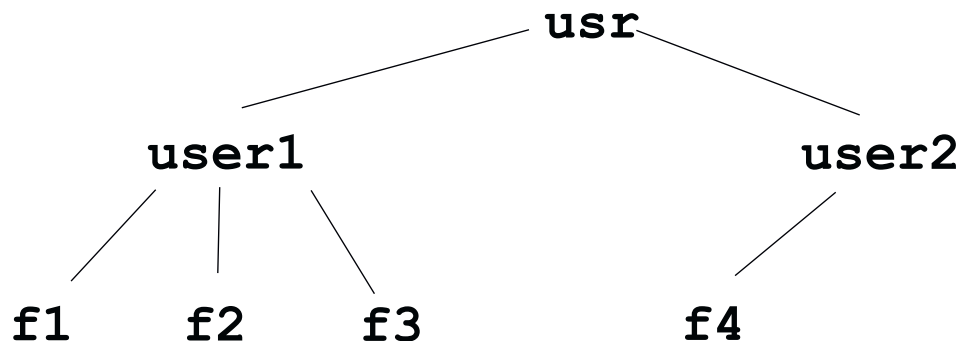
```
a global variable
a global variable
this is local
a global with new value
```

TD1: Linux et exercices

Ex1: Wildcard characters

- List the entries inside the folder **/usr/bin** whose name starts with the letter *m*.
- List the entries inside the folder **/usr/bin** whose name starts with the letter *m* and it has exactly 3 characters
- List the entries inside the folder **/usr/bin** whose name starts with the letter *m* and it has at least 3 characters
- List the entries inside the folder **/usr/bin** whose name starts with the letter *m* and it has an extension (postfix after .)

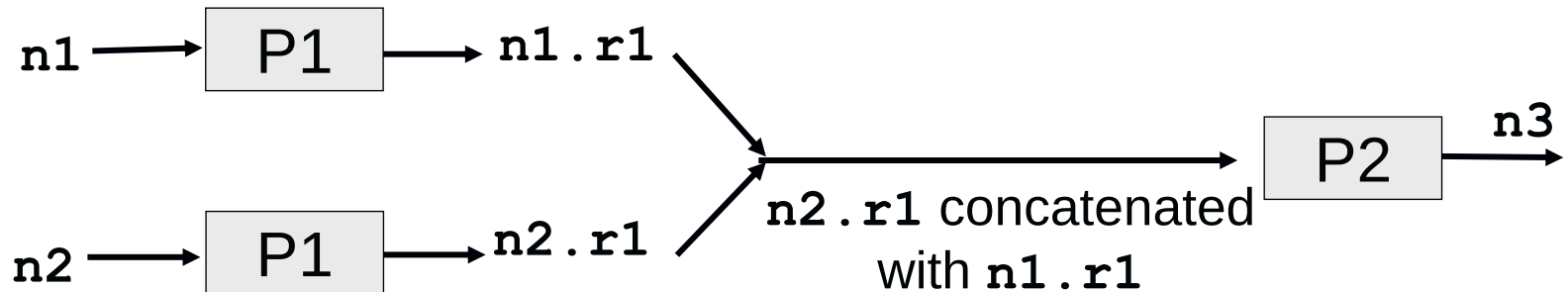
- Given the following file hierarchy



- **user2** is connected at his home folder
- Use the command **cat** :
 - Create a file **f5** with the following content
5 2 3 -1
 - Create a file **f6** by concatenating **f5** after **f4**
 - Add the file **f1** of **user1** at the file **f6**
 - Copy the file **f2** of **user1** as a new file **f7** of **user2**

Ex3: Redirections

- Create script with the name **execution** called with 3 arguments **n1 n2 n3** (which are the filenames) performing the following manipulations

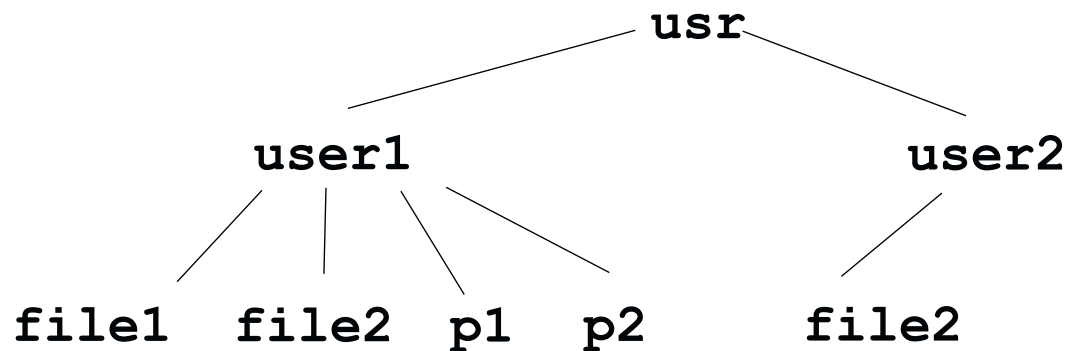


- **P1** and **P2** are programs that read from the `stdin` and print their result to `stdout`

Ex3: Redirections

- What happens when the following instruction is executed in the home directory of **user1**?

execution **file1** **../user2/file2** **fileN**



- Write an interactive version of the script **execution** that asks from the user to provide **n1**, **n2** and **n3**, instead of passing them as arguments

Ex4: Script arguments

- Create a script with the name **argument** using the command **expr** that prints one by one the arguments passed during the script call.

For instance, the call:

```
% argument one two three
```

Should print in the stdout:

```
3 arguments :
```

```
argument 1 - one
```

```
argument 2 - two
```

```
argument 3 - three
```

- Créez un script avec le nom "mysearch" en passant deux arguments lors de l'appel, c'est-à-dire "motif" et "nom de fichier". Le programme affiche soit "filename a le motif" soit "filename n'a pas le motif".
- Créez un script "mysearch2" avec un argument "filename" qui demande à l'utilisateur le motif à rechercher à l'intérieur du fichier jusqu'à ce que la fin du motif soit entrée par l'utilisateur.
- Créez un script "mysearch3" avec des arguments "f1" à "fn" qui effectue la même chose que "mysearch2", mais cette fois-ci, il recherche à l'intérieur des fichiers donnés en tant qu'arguments.
- **/dev/null** = trash

■ Décrivons l'effet des instruction suivantes :

- `find ./ -name "tp2" -exec {} < f_in > {}.out \;`
- `find /var/log -exec grep "www.athabasca.com" {} \;`
- `find . -name "rc.conf" -exec chmod o+r {} \;`
- `find /usr/local -type f -name "*.html".`

- Calculator